

Real-Time Communication Patterns

Polling · **Webhooks** · **SSE** · **WebSockets**

Pick the right pattern. Build apps that feel live without overengineering.

What we'll cover

Four patterns, three projects, one running scenario.

- The four real-time patterns and what each is for
- When to pick which (the decision tree)
- HTTP basics + JWT auth, refreshed
- Hands-on: 7 runnable example folders (server + client per pattern)
- Three full projects, including the LiveOrder capstone that composes all four
- Where these show up in AI apps (LLM streaming, MCP, voice agents)

Meet the cast

Every example below is told through these four. Same people, different patterns.

M
Maya
Backend developer

*Builds LiveOrder's backend.
Decides which pattern
for each feature.*

R
Raj
Hungry customer

*Orders biryani on his phone.
Wants live status
and instant chat.*

P
Priya
Restaurant owner

*Watches the kitchen tablet.
Gets notified instantly
when a new order pays.*

S
Sam
Delivery driver

*On a scooter with the food.
Chats with the customer
about buzzer codes.*

Each real-time pattern fits naturally in a moment of their day.
Maya's job: pick the right pattern for each feature so Raj's experience feels live.

Maya builds it · Raj uses it · Priya cooks it · Sam delivers it

The problem with plain HTTP

The client always speaks first. The server can never push.

HTTP - Request / Response, then the conversation ends

Stateless. Client always initiates. This is the limitation the next 4 patterns work around.



③ Connection closes (or kept-alive for the NEXT request, but each request is still independent)

Key idea

- Server cannot speak unless asked.
- Each request is independent - server has no memory between them.
- Every other pattern in this workshop exists to escape this limitation.

Every real-time pattern in this workshop exists to work around this one limit.

HTTP status codes that matter for real-time

Memorise these. You'll handle them in every pattern below.

CODE	MEANING	WHEN YOU'LL SEE IT
200	OK	successful poll; successful webhook receipt
201	Created	after a POST that creates something
202	Accepted	'Got it, processing async' - webhook receivers
204	No Content	long-poll timeout; nothing to send
304	Not Modified	ETag/If-None-Match - smart polling savings
401	Unauthorized	missing/expired token; bad webhook signature
404	Not Found	wrong URL or missing record
408	Request Timeout	long-poll fell past server limit
429	Too Many Requests	polling too fast or rate-limited
500	Internal Server Error	bug; crashed dependency
502	Bad Gateway	proxy got nothing useful from backend
503	Service Unavailable	maintenance, overload; webhook senders retry

Stateless vs stateful, per pattern

HTTP itself is stateless. Some patterns add stateful connections on top.

PATTERN	CONNECTION MODEL	STATEFUL?
Polling (short)	new HTTP request per poll	Stateless
Polling (long)	one held HTTP request	Stateless on server, in-flight on client
Webhook receiver	separate HTTP request per event	Stateless
SSE	one long-lived HTTP connection	Stateful at the transport level
WebSocket	one persistent connection, both directions	Stateful

Stateful = costs memory per client. 100K SSE/WS connections = ~10 GB. Plan accordingly.

The four patterns at a glance

Mental model first. Wire format and code in the next sections.

POLLING

Are we there yet?

Client repeatedly asks the server on a timer. Crude. Sometimes the right answer.

WEBHOOKS

*Don't call us,
we'll call you*

External server POSTs to your URL when something happens. Server-to-server.

SSE

*Tune in.
Server broadcasts.*

Server holds one HTTP connection and pushes events. Browser-native. One-way.

WEBSOCKETS

Open a phone line

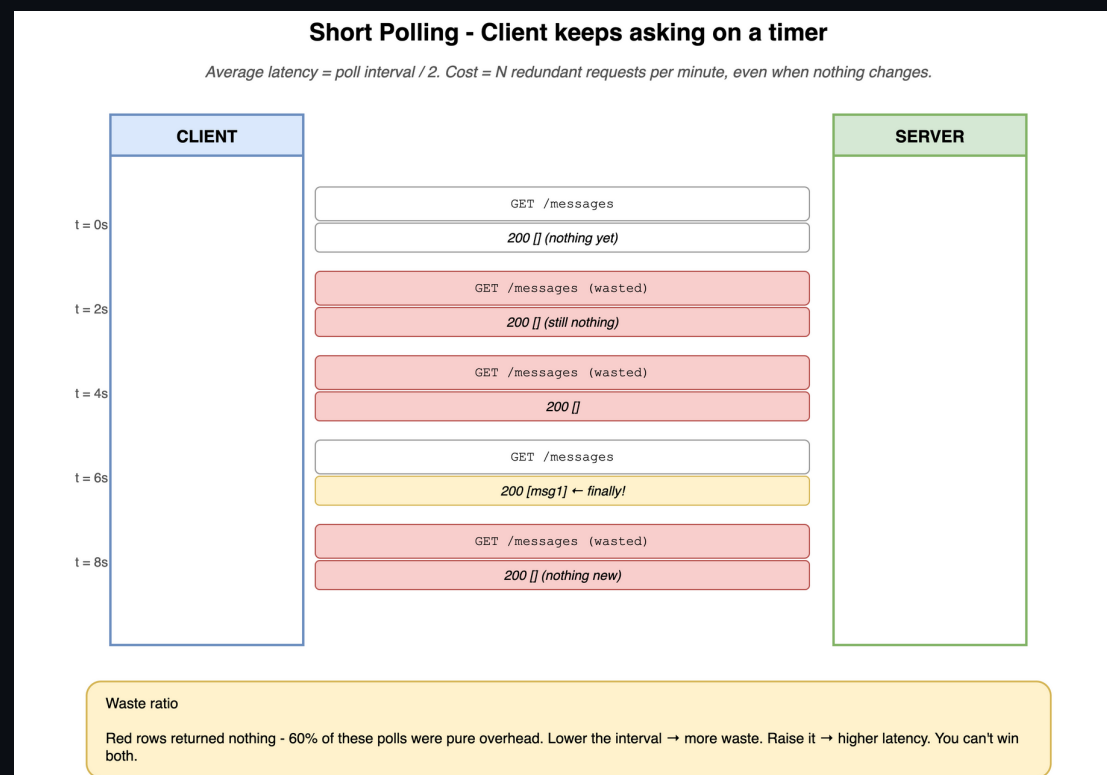
Persistent, bidirectional, low-overhead. Use only when you really need two-way.

SECTION

Polling

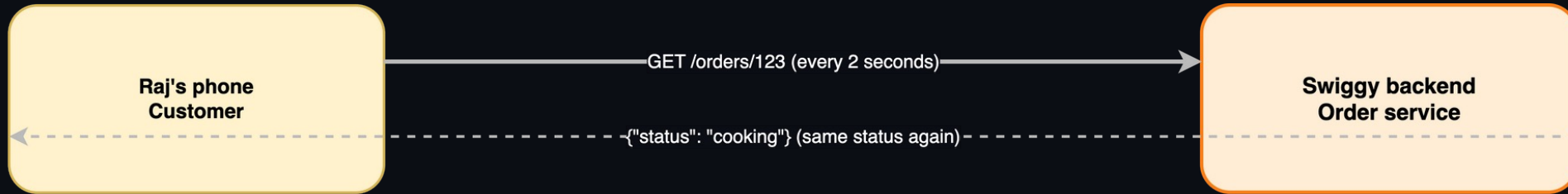
Short polling - the protocol

Client asks on a timer. Server immediately returns whatever it has.



Raj refreshing his Swiggy order

The kid in the back seat. 'Are we there yet?' Most refreshes return the same status.

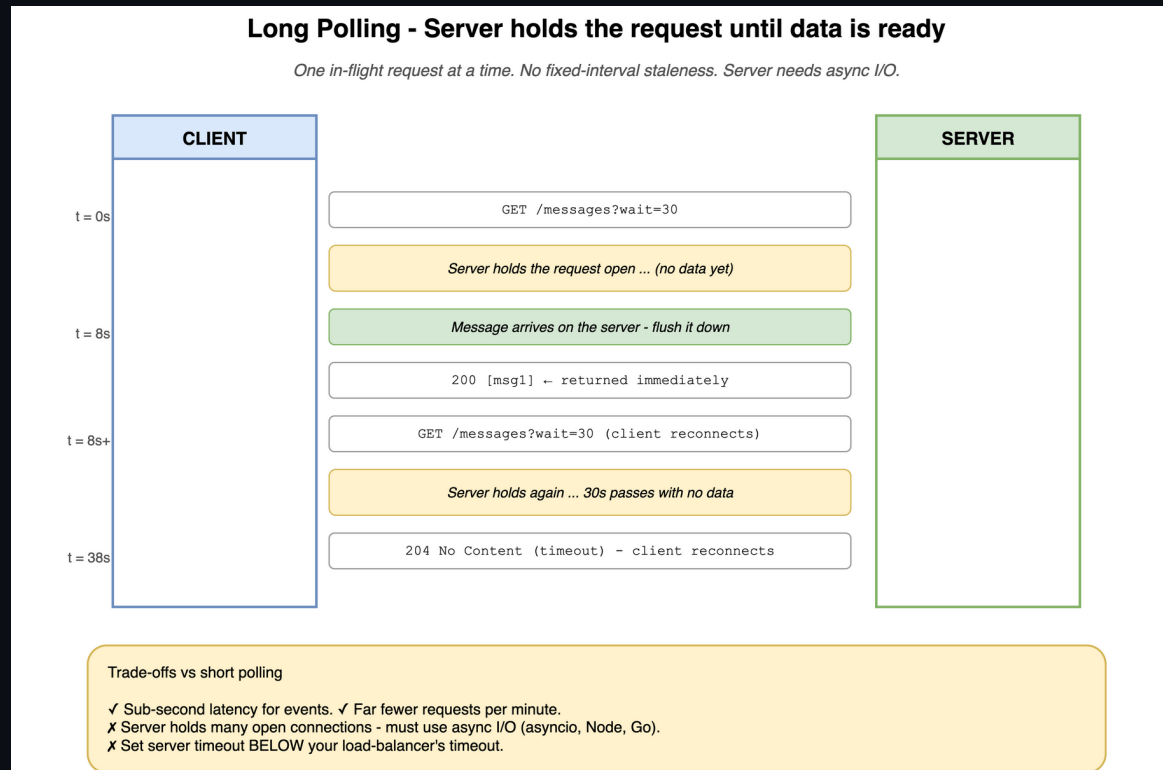


Of 25 polls during an order:
about 5 catch a real status change (placed -> confirmed -> cooking -> out for delivery -> delivered)
the other 20 return the same status as last time - that's 80% wasted requests.

When polling is fine
Status updates that don't need sub-second latency. Small client counts. Backends that don't push.

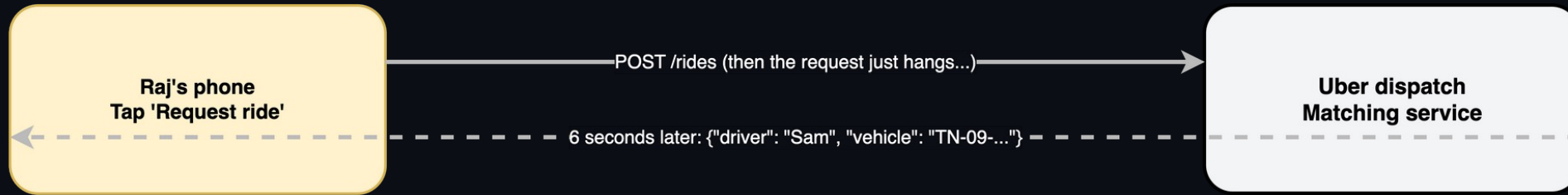
Long polling - the protocol

Server holds the request open until data is ready (or timeout). One held connection per client.



Raj waits for an Uber driver

One held request. The server replies the moment a driver accepts. No wasted polls.



ONE request instead of many
Short polling would have fired ~6 requests at 1s interval, 5 of them empty.
Long polling sent 1, got the answer the moment a driver accepted, no waste.

Watch out
Server must use async I/O (one held connection per rider). Server's timeout **MUST** be shorter than the load-balancer's idle timeout, or the LB cuts the connection first.

Making polling cheaper without changing patterns

Six tricks that cut waste 10x. Use them ALL before you reach for SSE.

1. Cursor / since= parameter

Send the last-seen id so the server returns only deltas.

2. ETag + If-None-Match

Server returns 304 with empty body when nothing changed. Bandwidth -> ~zero.

3. Exponential backoff

If five polls in a row return empty, slow down. Reset on activity.

4. Jitter

Add +-10% random offset so 10K clients don't sync up and DDoS you.

5. Pause when invisible

Page Visibility API. No point polling for a tab no one's looking at.

6. Conditional auth caching

Don't hit your auth DB on every poll. JWT verify in-process is fine.

Polling bugs you'll hit, and the fix

Every team rediscovers these. Save yourself the outage.

'Data never updates' even though server has new data

Browser or CDN cached your GET. Add Cache-Control: no-store, or ?t=<ts> cache-buster.

Long-poll times out at 30s even though you set 60s

A proxy or LB has a shorter idle timeout. Set server < LB always.

Worked in dev, exploded in prod

Dev had 1 user; prod has 10K. Add per-endpoint req/sec and DB query metrics; you'll see the N+1.

Polling continues after tab closes

Forgot to clearInterval on unmount/visibilitychange. Always pair start with stop.

UI shows duplicate notifications on every poll

Missing cursor; you're re-fetching the whole list. Add ?since=<last_id>.

When polling is the right call (and when it isn't)

It's not always wrong. Just usually mis-used.

Use polling when

- Updates are rare and a few seconds stale is fine
- You're calling someone else's API that only supports REST
- Total clients < 100; engineering cost matters more than waste
- You need to ship in an afternoon
- Status checks for slow background jobs (batch reports, fine-tunes)

Don't poll when

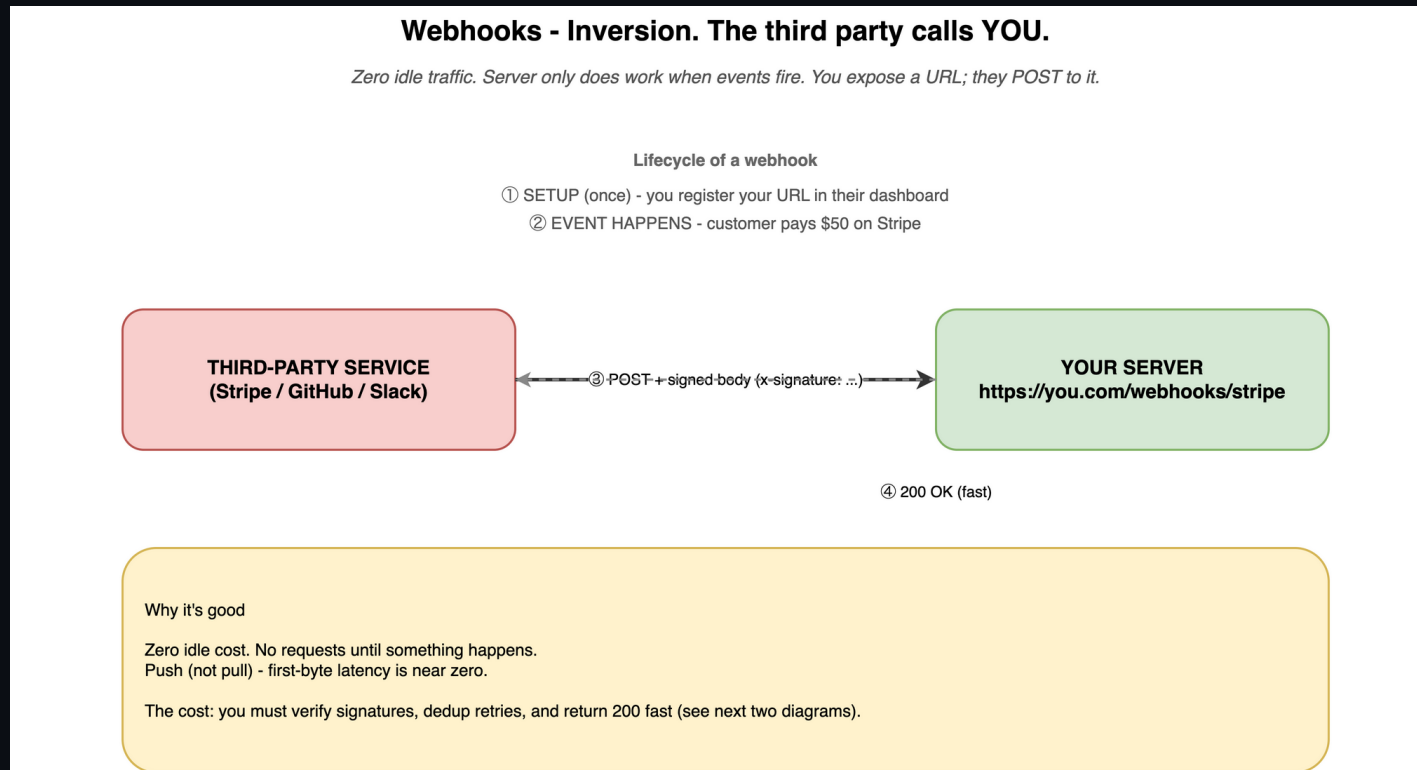
- The UI must feel live (chat, typing, multiplayer, game)
- You have 10k+ mostly-idle clients (millions of empty requests)
- You'd be polling at 10+ Hz - just use SSE
- You're hitting someone else's rate-limited API quota
- The source already supports webhooks; just use those

SECTION

Webhooks

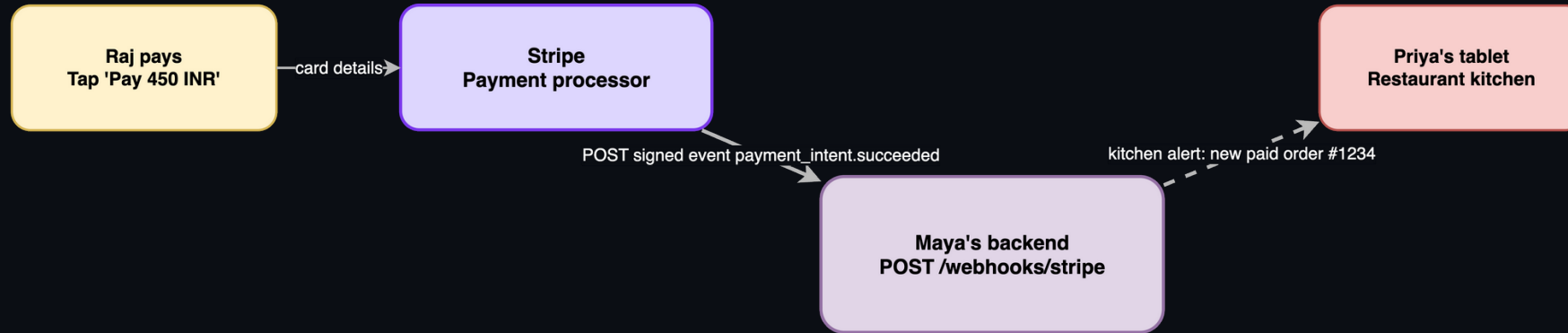
Webhook - the protocol

Third-party server POSTs to YOUR URL when something happens. Zero idle traffic.



Stripe pays for Raj's biryani

Stripe is the source of truth. They tell Maya's backend when the card cleared.



Maya's backend has 5-10 seconds to respond 200. She verifies the HMAC, writes to a queue, returns 200. The kitchen notification happens asynchronously.

The 4 webhook rules

1. Verify the signature (HMAC-SHA256 of the body with a shared secret).
2. Dedup by event id - Stripe will sometimes deliver the same event twice.
3. Return 200 fast - queue real work; don't do it in the handler.
4. Return 200 (not 5xx) for events you don't care about; 5xx triggers retries.

The four rules of webhook receivers

Skip any one and you'll have very interesting outages.

1. Return 2xx fast

Don't do work in the handler. Verify, queue, return 200. Stripe times out at 5-10 seconds.

2. Verify the signature

HMAC-SHA256 of the body, constant-time compare. Without this, anyone can fake events.

3. Dedup by event id

Senders sometimes deliver twice. Without dedup, you'll double-charge cards.

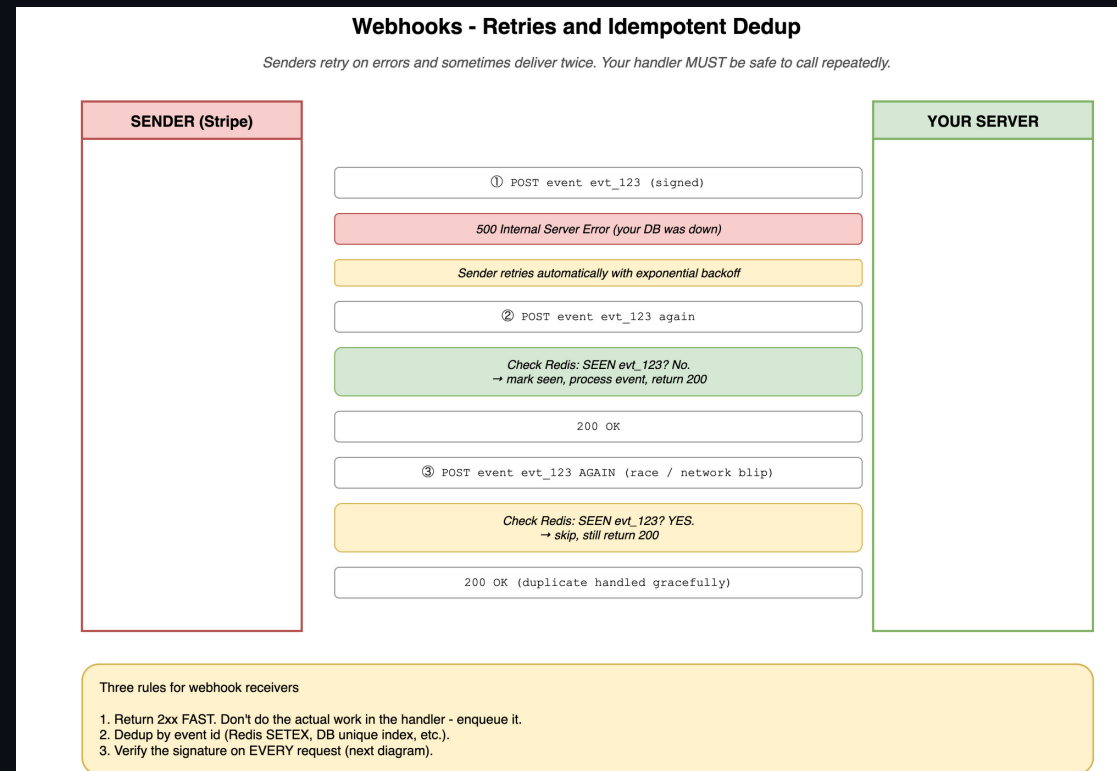
4. Return 200 (not 5xx) for events you don't care about

5xx triggers retries forever. Log + 200 is the right answer for noise.

WEBHOOK

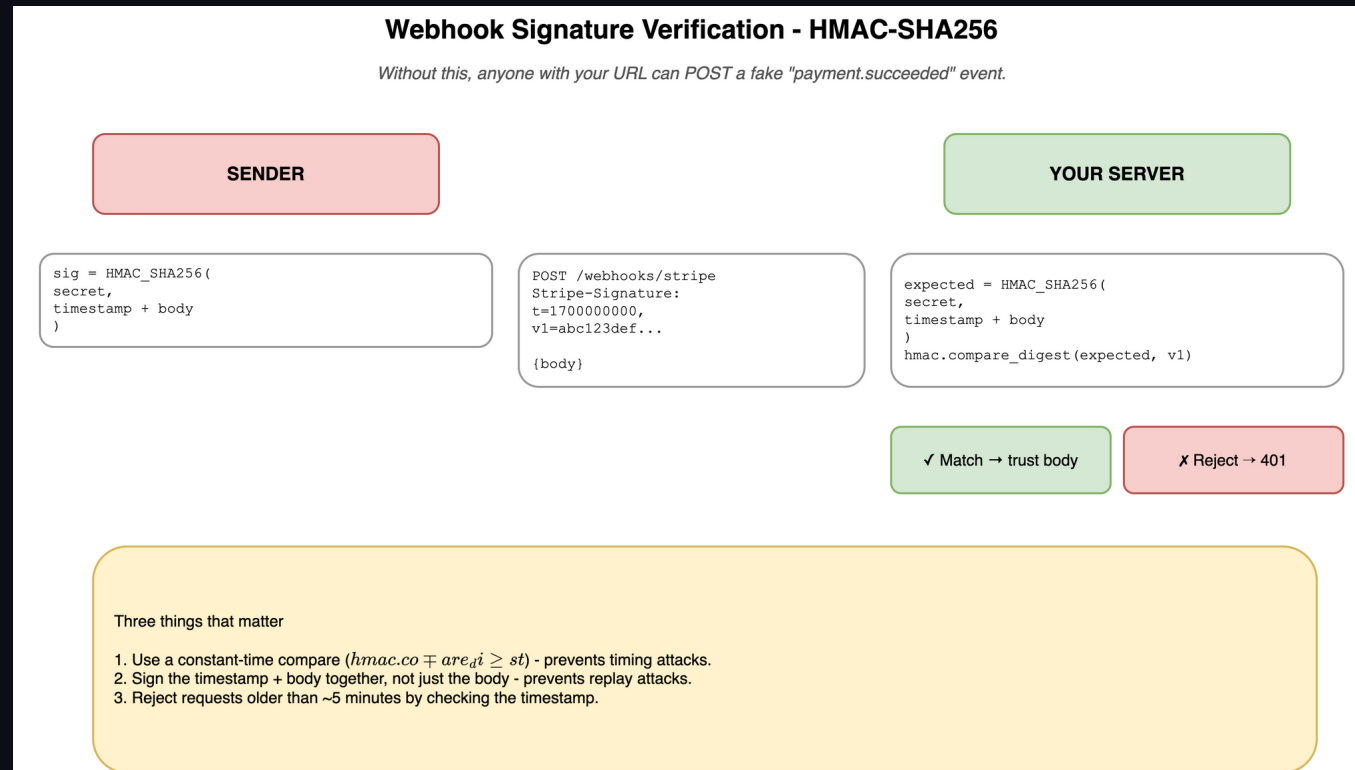
Retries and idempotent dedup

Same event id arriving twice is normal. Be ready.



Signature verification - HMAC-SHA256

The one thing standing between your webhook URL and fake events.



WEBHOOK

Testing webhooks locally

Stripe can't reach localhost:8000. Here's how to bridge.

webhook.site

look only

Visit <https://webhook.site>, get a unique URL, paste into Stripe. Sees every request in a dashboard. Can't run YOUR code against it but great for inspecting payloads.

ngrok / Cloudflare Tunnel / Tailscale Funnel

tunnel

ngrok http 8000 -> public URL forwarding to localhost. Combined with the ngrok web inspector (localhost:4040) you can see + replay every request.

Stripe CLI (or provider equivalent)

real events, no tunnel

stripe listen --forward-to localhost:8000/webhooks/stripe. Forwards real test webhook events from your Stripe account straight to localhost. Handles signature setup.

If you're the **SENDER** (offering webhooks to customers)

The other side of the table. Same rules, mirrored.

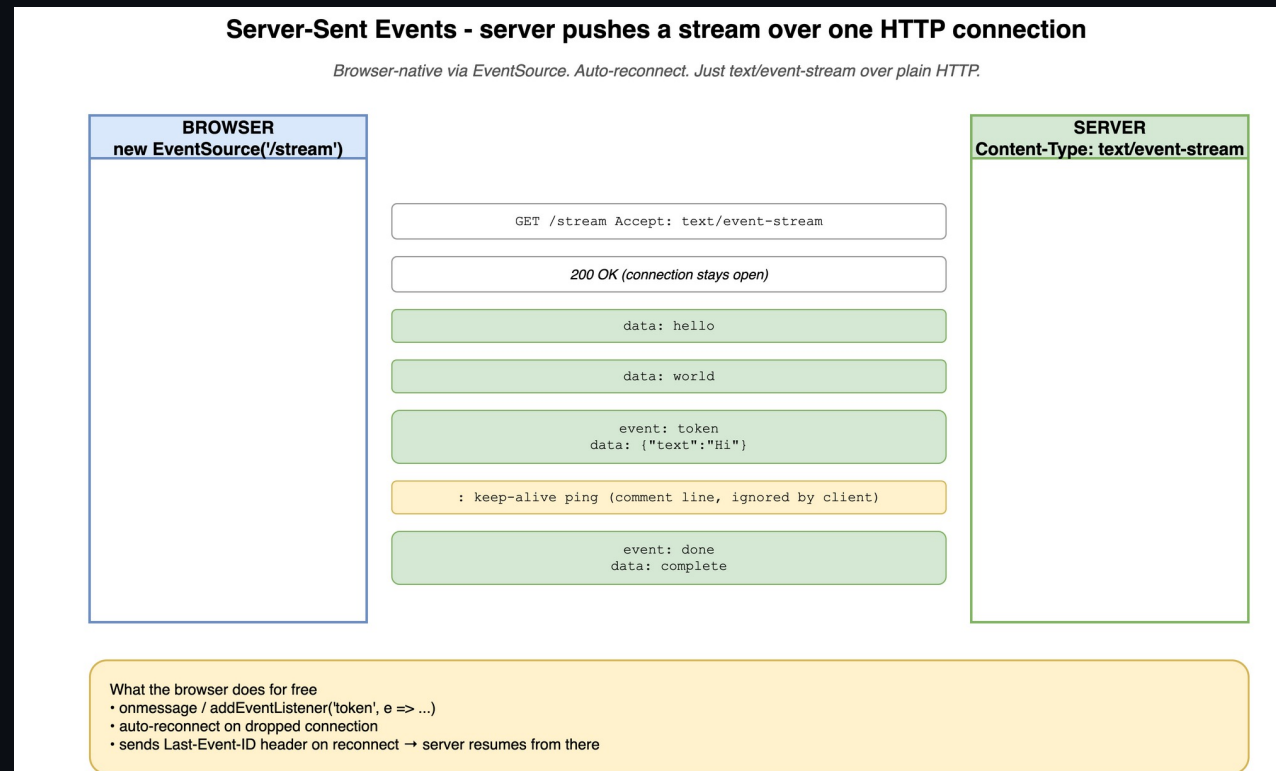
- Make registration first-class. UI + API to register URLs, multiple URLs per account, filter by event type.
- Sign every payload. Per-customer signing secret. Document the algorithm and rotation flow.
- Include a timestamp in the signed payload (defeats replay attacks).
- Retry with backoff. Standard schedule: 1m, 5m, 15m, 1h, 6h, 12h. Give up after ~24h, mark endpoint disabled.
- Provide a 'Send test event' button in the dashboard - your customers will use it constantly.
- Maintain a delivery log: last 100 deliveries, status codes, response times, response bodies. Invaluable for support.
- Build internal rate limits. A bug that fires 1M duplicate webhooks tonight = 1M customer outages.

SECTION

Server-Sent Events

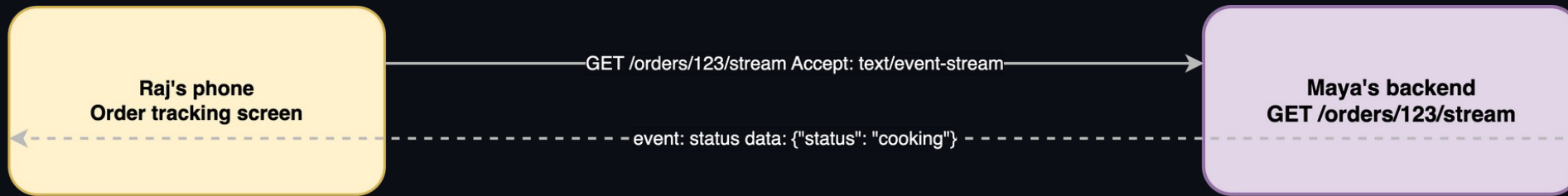
SSE - the wire format

One HTTP connection. Server streams labelled events; client reads as they arrive.



Raj watching his order status live

Server pushes 'paid -> cooking -> out for delivery -> delivered' as events. Zero polling.



Same one HTTP connection. As the order progresses, the server pushes an event for each transition:

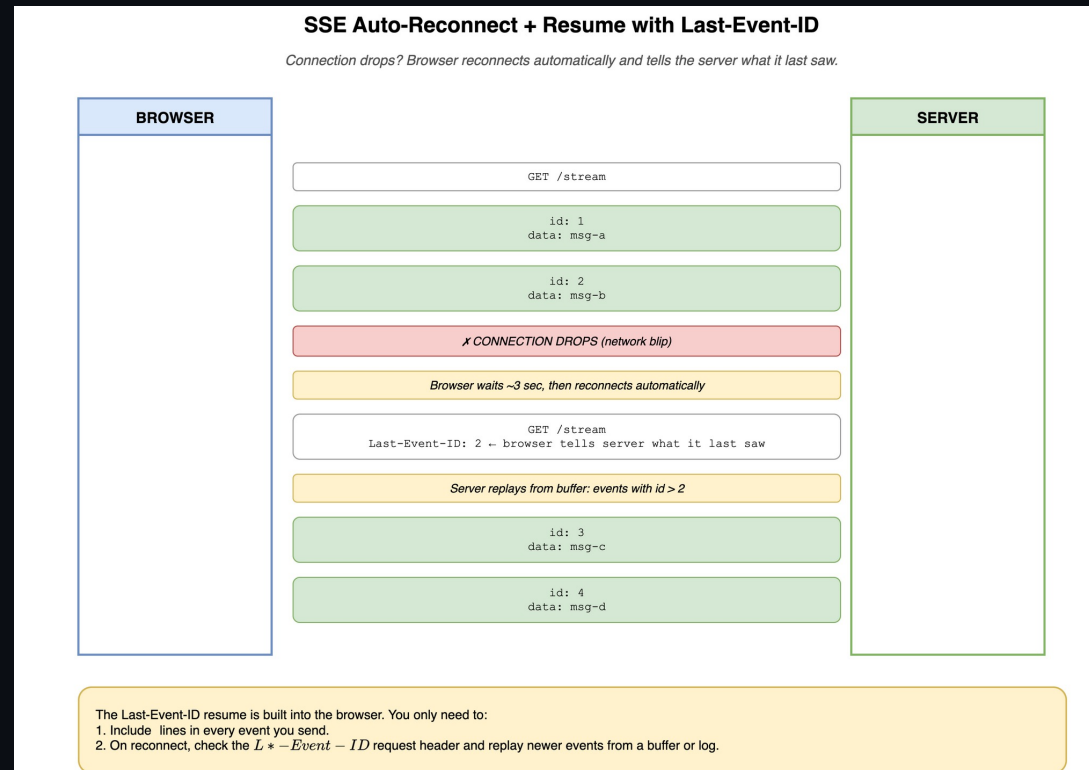
paid -> restaurant_confirmed -> cooking -> out_for_delivery -> delivered

Raj's phone sees each update within ~50ms. Zero polling.

Bonus: if the connection drops (subway tunnel), the browser auto-reconnects and sends Last-Event-ID. The server picks up exactly where it left off.

Auto-reconnect with Last-Event-ID

Connection drops? Browser reconnects and tells the server what it last saw. Server resumes.



Streaming an LLM response (the ChatGPT pattern)

Backend proxies OpenAI's SSE stream as SSE to the browser. Same wire format both sides.



Why proxy through Maya's backend instead of letting the browser call OpenAI directly?

1. The API key never reaches the browser.
2. Maya can log/filter/rate-limit prompts.
3. Cookies and domain stay simple - the browser only talks to liveorder.app.

The SSE grammar - all of it

Five fields plus a blank line. That's the protocol in full.

event: <event-type>	optional, default 'message'. JS addEventListener key.
id: <event-id>	optional. Browser remembers; sends as Last-Event-ID on reconnect.
retry: <ms>	optional. Tells browser the reconnect delay.
data: <line of payload>	can repeat; multiple data: lines join with '\n'.
:<comment>	ignored by client. Use to send keep-alive pings.
<- BLANK LINE ends this event	

Examples

```
-----
data: hello                                # simplest possible event

event: token                               # named event with JSON payload
data: {"text": "Vada"}

id: 42                                     # with id - enables Last-Event-ID resume
event: chunk
data: {"text": " Pav"}

: keep-alive ping                         # comment - keeps proxies from closing
```

SSE gotchas every team hits

All recoverable. Knowing them up front saves the all-nighter.

All events arrive at once at the end

A proxy or middleware is buffering. nginx: 'proxy_buffering off' or header 'X-Accel-Buffering: no'. Disable gzip on event-stream endpoints.

Connection dies after 60s of silence

Idle-killer somewhere (LB, proxy, mobile network). Send ': ping\n\n' every 15-20 seconds.

Browser shows 'reconnecting' forever

Server is returning a non-200 status. EventSource keeps retrying but never succeeds. Check server logs and CORS headers.

Authorization header doesn't work

EventSource doesn't allow custom headers. Use cookies (best), short-lived URL token, or the event-source-polyfill library.

Sometimes I get duplicate events after reconnect

Server replay is using $\geq$ instead of $>$ the Last-Event-ID. Use strict greater-than.

Closing the tab leaves connection open server-side

Normal - server only finds out at next heartbeat write. Clean up subscribers in your generator's 'finally' block.

You've used SSE thousands of times

Anywhere you've watched text appear word-by-word in 2023+.

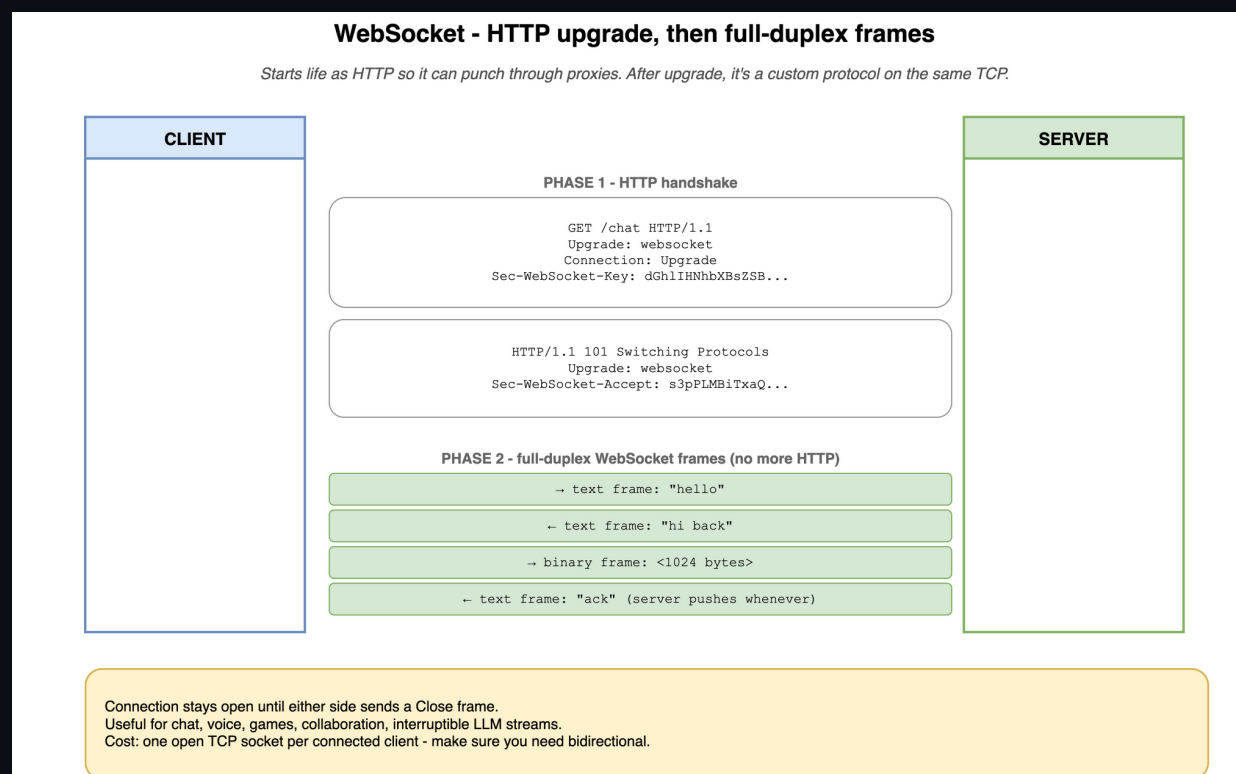
- ChatGPT, Claude.ai, Gemini chat - the typing animation IS SSE
- Cursor, GitHub Copilot Chat, Continue.dev - same pattern
- OpenAI / Anthropic / Mistral SDKs with stream=True - SSE clients
- Vercel AI SDK useChat hook - SSE client + parser
- Vercel / Render / Netlify deploy logs streaming in browser - SSE
- MCP servers (Claude Desktop tools) - HTTP+SSE / Streamable HTTP transport
- LangSmith and LangFuse trace viewers - SSE

SECTION

WebSockets

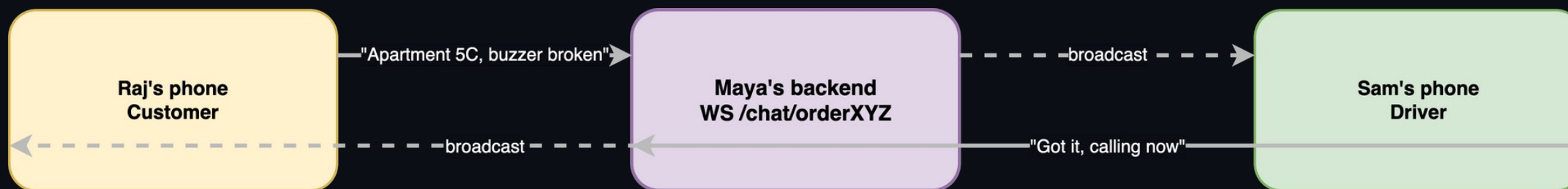
WebSocket - the handshake

Starts as HTTP. Upgrades to a custom frame protocol on the same TCP. Both sides talk freely.



Raj and Sam chat about the buzzer

Both can send anytime. Typing indicators are free. One TCP connection per chat.

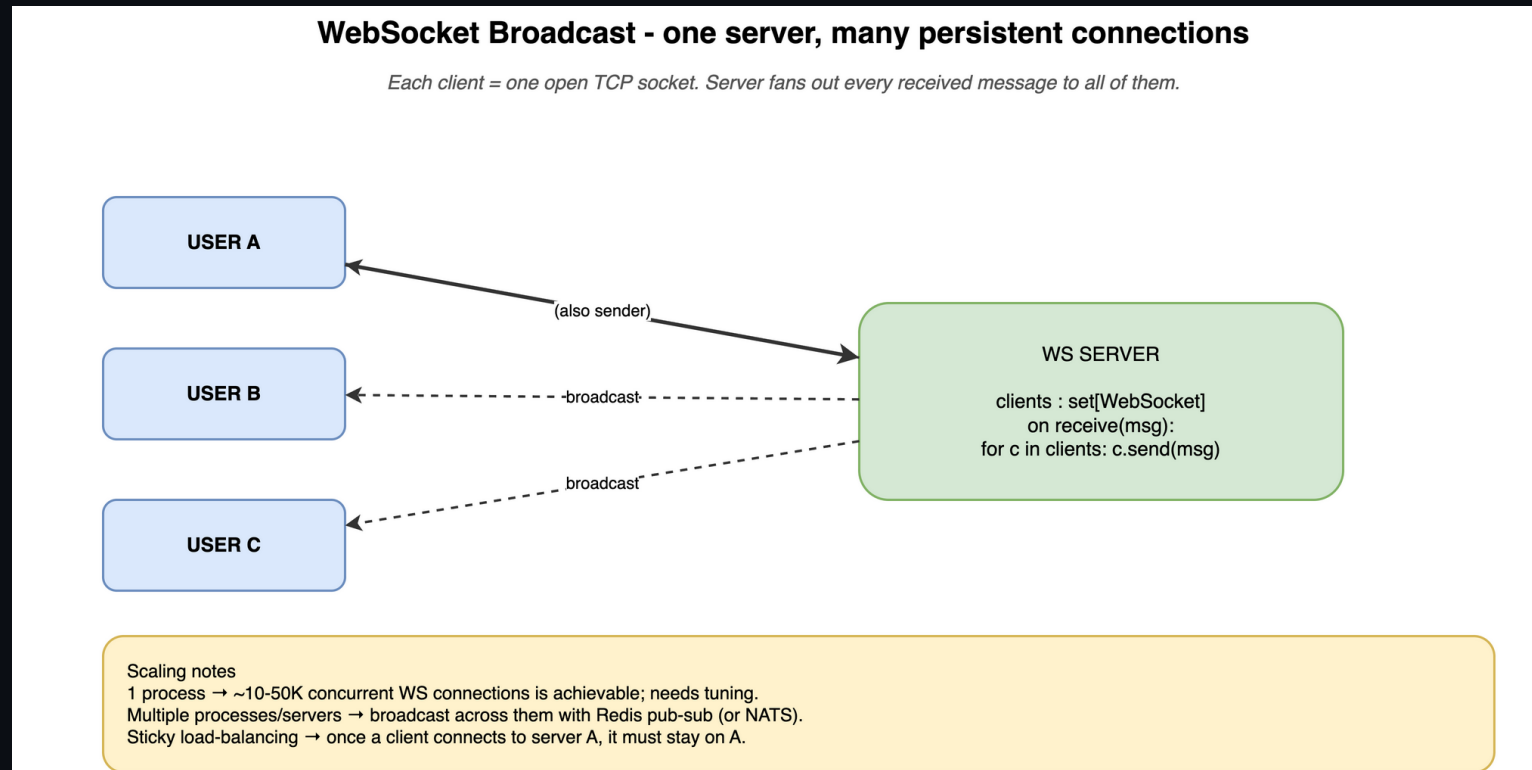


Polling: Raj would refresh every few seconds (laggy + wasteful).
SSE: Sam could push to Raj, but Raj couldn't push back - dead-end for chat.
WebSocket: both ways, low overhead per message, made for this.

The cost: one open TCP connection per connected person. 50K users = real RAM. Use WebSocket only when truly bidirectional. SSE is fine for one-way streaming.

Broadcast topology

Server tracks connected clients and fans messages out. Across processes, use Redis pub-sub.



WebSocket scaling - what changes at each tier

Each level adds infrastructure. Know which one you're at.

Up to ~100

Anything works

One process. Default config. Pick by ergonomics.

100 - 10K

Tune the basics

Async I/O required. `ulimit -n 65535`. Heartbeat every 20s. LB idle timeout to 5 min.

10K - 100K

Pub-sub backbone

Multiple processes/instances. Redis or NATS pub-sub for cross-server broadcasts. Sticky LB sessions. Connection metrics.

100K - 1M

Specialised tools

Phoenix Channels (Elixir), Centrifugo, Ably, Pusher, PartyKit. Kernel tuning. Separate WS gateway tier from app servers. Edge delivery (Cloudflare/Fastly).

Over 1M

Specialised team

You're past blog-post territory. Hire someone who's done it before. This is a full ops surface.

WebSocket bugs you'll hit, and how to ship around them

Most production WS pain comes from these.

No reconnect logic on the client

Network blips silently kill your client forever. Always write a reconnecting wrapper with exponential backoff + jitter.

No heartbeat

Half-open TCP (network died but TCP doesn't know) looks alive until you write. Send `{type:'ping'}` every 20-30s; force reconnect if no pong.

Server is sync (Flask sync, Django pre-Channels)

Exhausts worker pool at ~50 connections. WS needs async (asyncio/uvicorn, Node, Go) - period.

Custom auth via browser headers

Browser WebSocket API does NOT allow custom headers. Use cookies, a subprotocol-as-token, or a short-lived URL token (`ws://...?ticket=xxx`).

No backpressure handling

Slow client + fast server = OOM. Bound your send queues; close clients that fall behind.

Works on dev (one server) breaks on prod (LB + multiple servers)

Sticky sessions on LB; pub-sub backbone (Redis) so broadcasts cross processes; check LB actually supports Upgrade header.

SSE vs WebSocket - when each one wins

Default to SSE. WebSocket only when you NEED bidirectional.

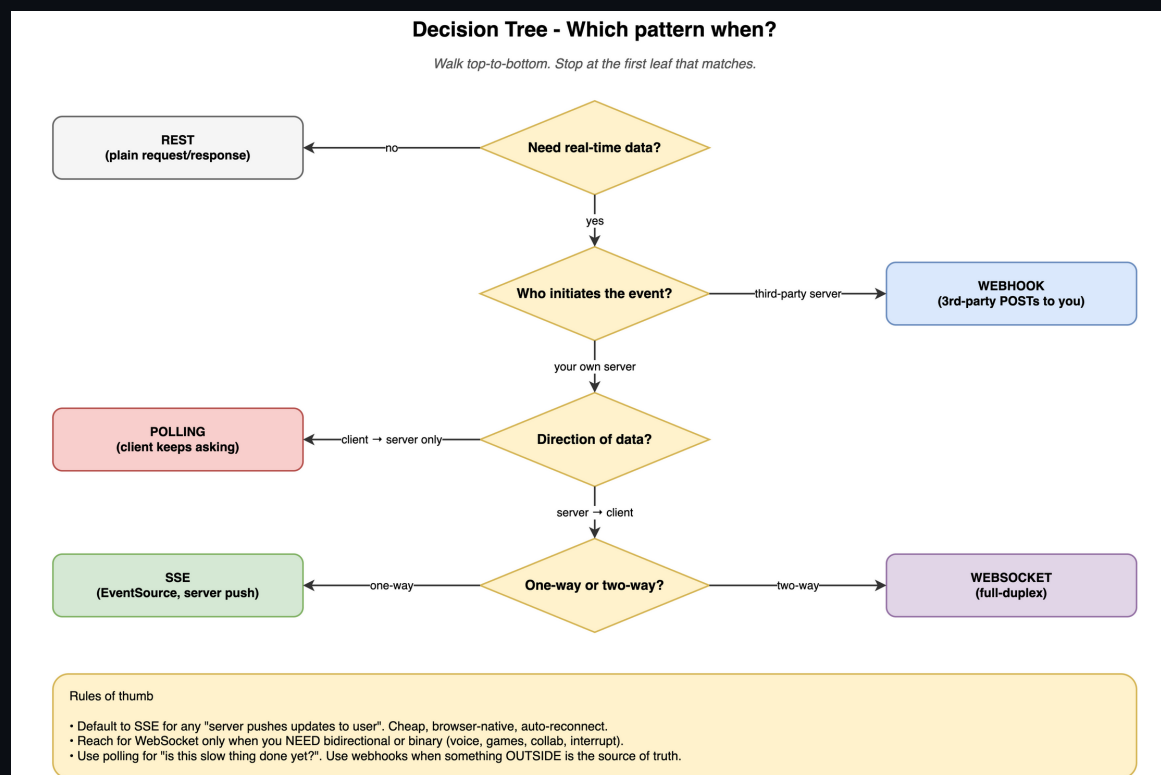
	SSE	WebSocket
Direction	Server -> client only	Both ways
Protocol	Plain HTTP	HTTP upgrade to ws://
Browser API	EventSource (built in)	WebSocket (built in)
Auto-reconnect	Yes, free	No, you write it
Binary support	No (text only)	Yes
Through proxies	Almost always works	Sometimes blocked
Best for	LLM tokens, status, logs	Chat, voice, games, collab

SECTION

Decision Matrix

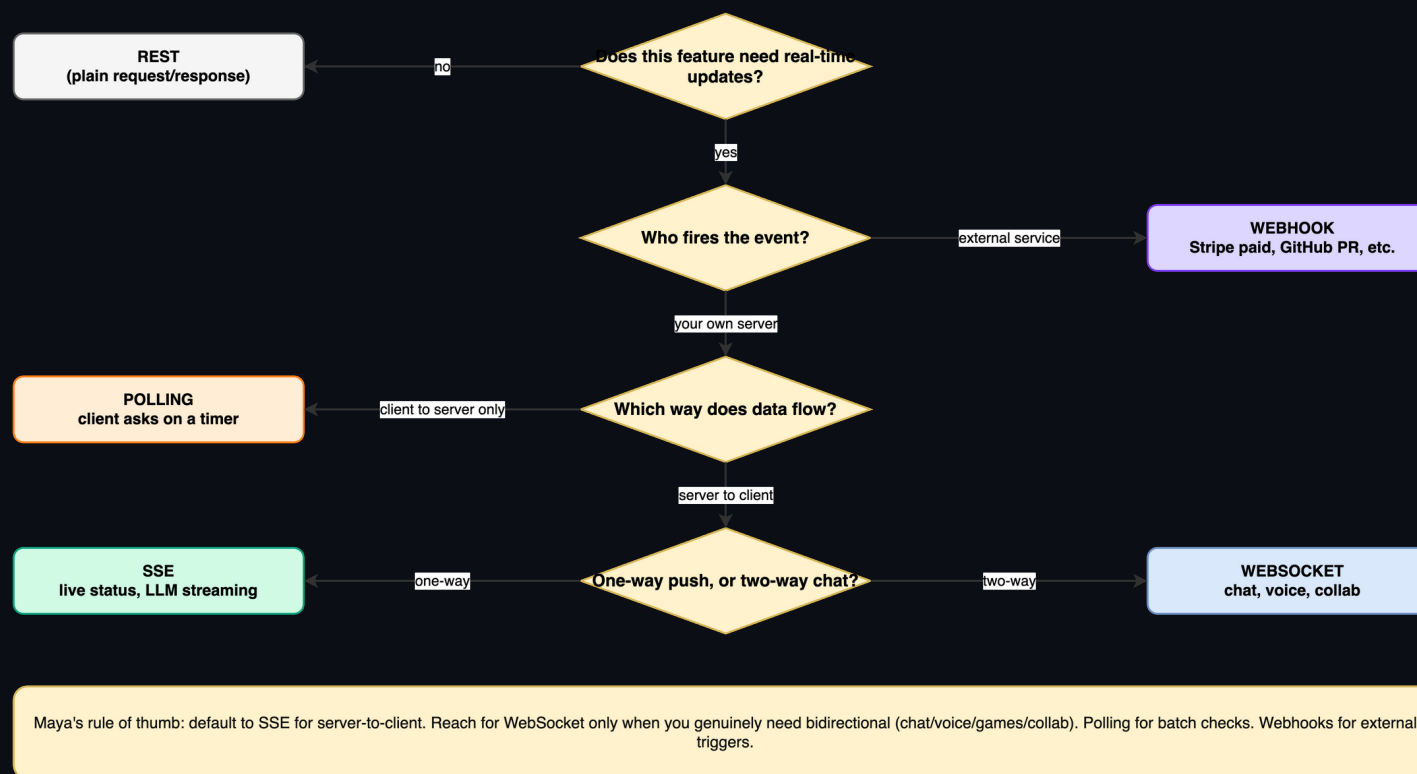
The decision tree - generic

Walk top to bottom. The first leaf that matches your data flow is the answer.



Maya picking the pattern for a new feature

Same tree, framed as Maya's design checklist. Same answer, more memorable.



'I'm building X' - 8 quick recipes

Common features and the pattern that fits each.

ChatGPT-style typewriter	-> SSE	Browser-native, auto-reconnect, simple
Slack-style chat	-> WebSocket	Both directions, low latency, typing indicators
Stripe payment notification	-> Webhook	Stripe is the source of truth; let them call you
Live stock ticker dashboard	-> SSE	One-way, many subscribers, no client input
Multiplayer browser game	-> WebSocket	Sub-100ms two-way, binary
Voice agent (audio in + out)	-> WebSocket	Bidirectional binary; SSE can't carry binary
'Is my batch report done?'	-> Polling	Low frequency, simple, OK to be a few seconds late
MCP server tool calls	-> SSE	One call -> many progress events; perfect fit

AI scenario - the agent kicks off a long task

Research agent takes 3-5 min. Three valid patterns to surface progress.

Polling

POST /jobs returns {id}. UI polls /jobs/{id} every 5s. Spinner -> done toast. Simplest, no progress detail.

SSE for progress

POST /jobs returns {id}. UI opens /jobs/{id}/stream. Server pushes 'searching web', 'reading 3 docs', 'writing'. Best UX - feels fast even when slow.

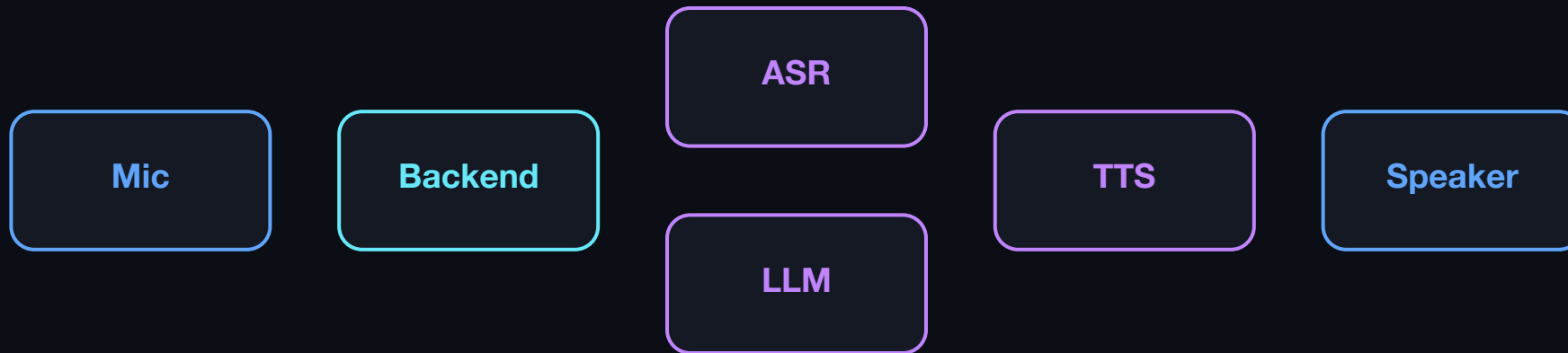
Webhook callback

When the worker is external (OpenAI Batch, your own remote service). Pass callback_url; remote service POSTs when done. Best for fully async.

Rule of thumb: SSE for progress within seconds. Webhook callback when the wait is hours.

AI scenario - voice or interruptible streaming

Microphone in, speaker out, mid-stream interrupt. Only WebSocket fits.



Audio frames flow both ways over one WebSocket · interrupting sends a 'stop' upstream while audio still streams down

SSE one-way (server -> client). Mic input has nowhere to go.

Polling latency is way too high; users would hear stutters.

Webhooks server-to-server only; no path from a user's microphone.

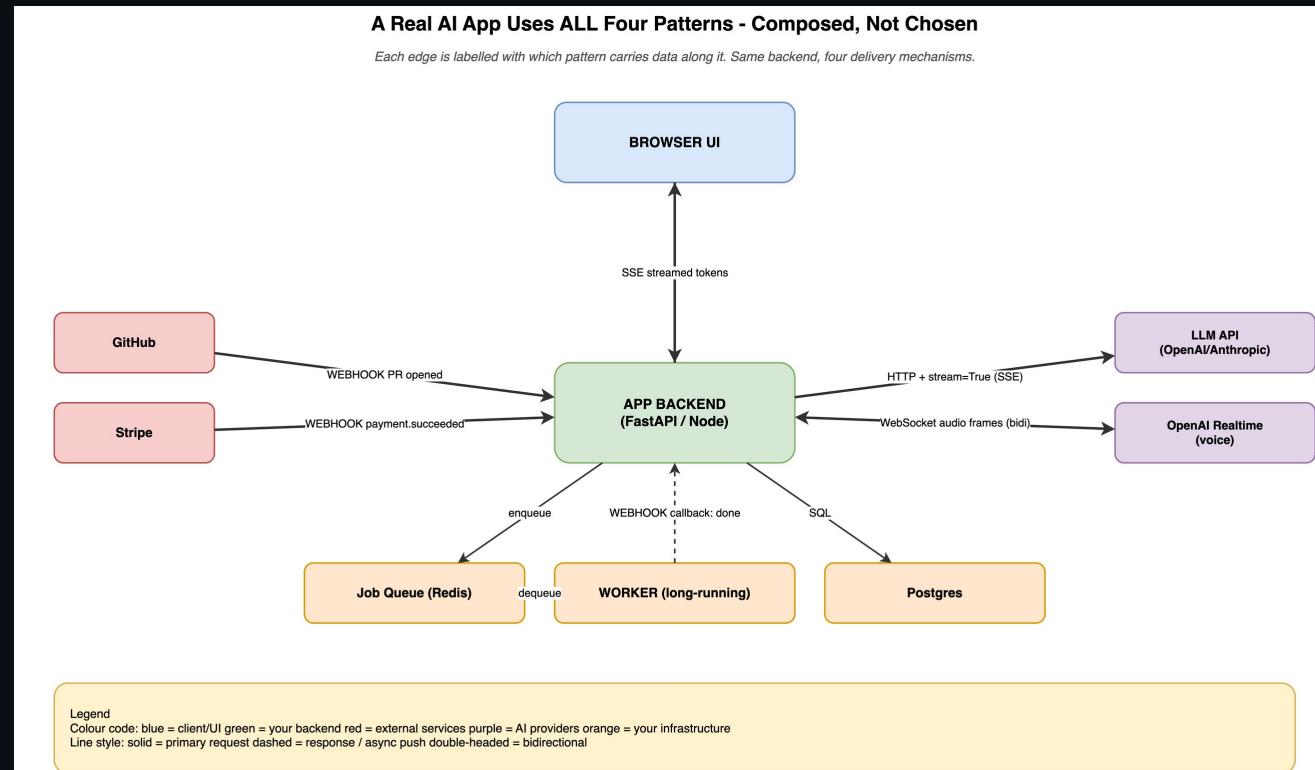
WebSocket binary + bidirectional + low overhead. Used by OpenAI Realtime, ElevenLabs, Vapi, Retell.

SECTION

All Four, Together

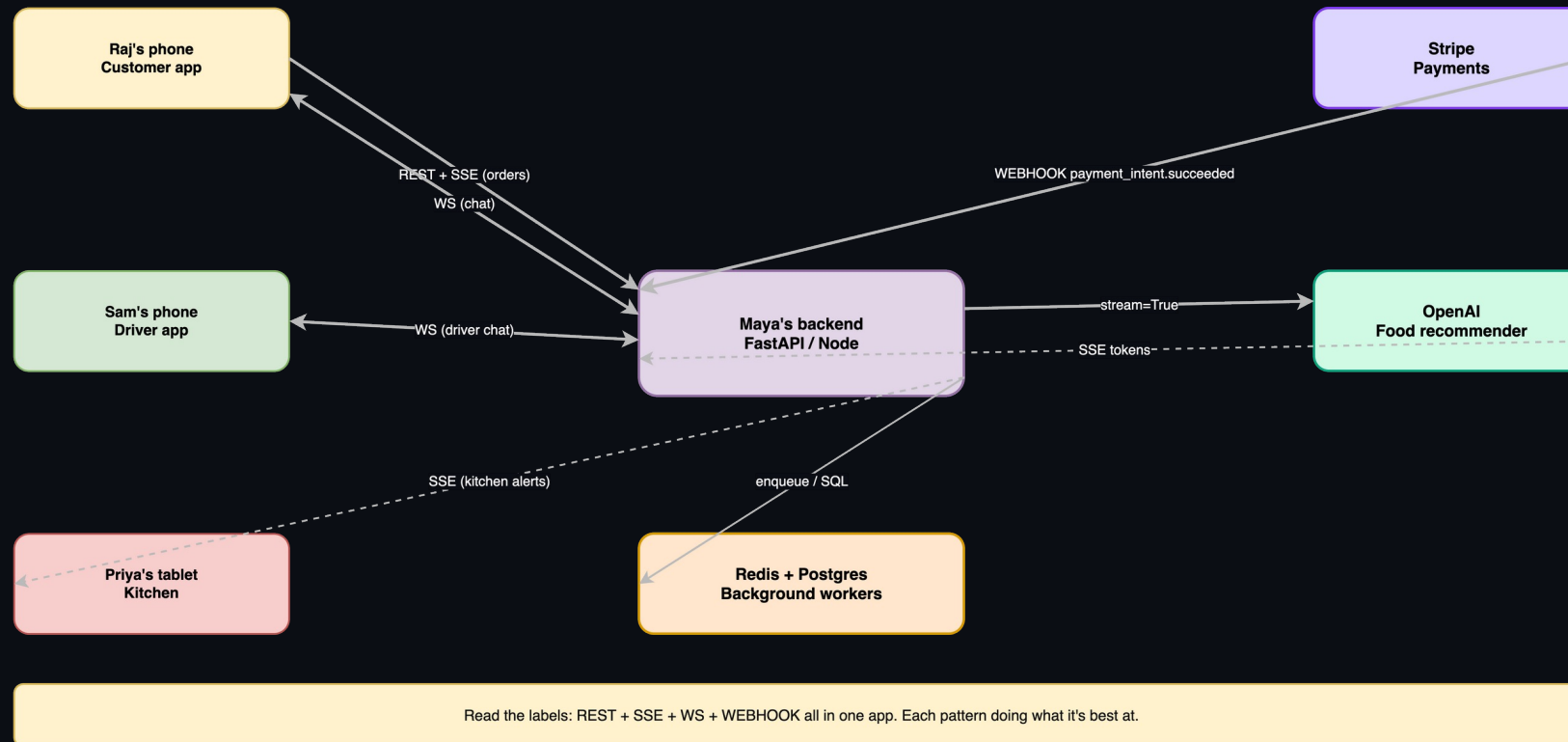
A real AI app uses all four

Generic architecture. Each edge labelled with which pattern carries it.



LiveOrder - all 4 in Raj's order journey

Same composition, our personas. This is Project 3 in the repo.



MCP - the transport

Claude calls tools over HTTP+SSE / Streamable HTTP. One request -> many progress events.

MCP (Model Context Protocol) - runs on SSE / Streamable HTTP

One tool call → many events (progress, intermediate results, final answer). Exactly the shape SSE is built for.

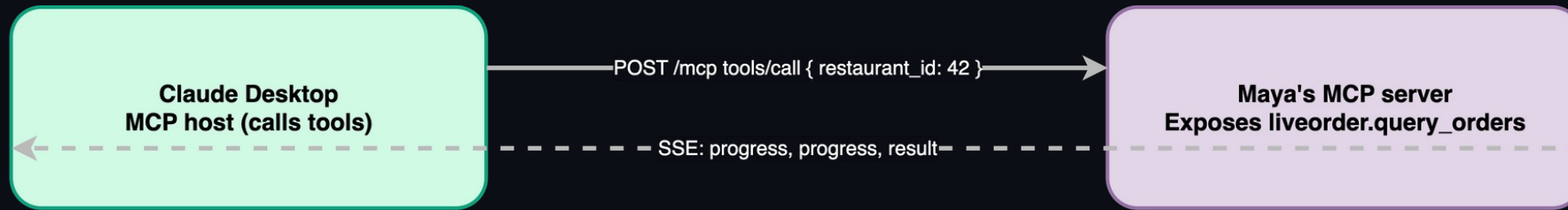


Why SSE for MCP?

- Tool calls take time → progress streaming is a natural fit.
- One request can produce many events (search → 10 results, then summary).
- Standard HTTP - no WebSocket upgrade, easy to host anywhere.
- MCP transports: 1) stdio (local processes) 2) HTTP + SSE 3) Streamable HTTP (current standard).

Maya exposing LiveOrder data to Claude

Maya's MCP server lets Claude query LiveOrder. The transport is SSE you've already learned.



Why SSE for MCP?

Tool calls take real time (DB queries, web fetches, summarising).
One tool call typically produces many progress events.
Standard HTTP - no WebSocket upgrade. Easy to host anywhere.
Resumable via Last-Event-ID on reconnect.

From the user's perspective, Claude asks 'how many veg orders did Priya's restaurant have last week?' and gets a real answer that came from Maya's database. The MCP transport is SSE the whole way.

SECTION

Workshop Kit

Three projects to fork

Each runs locally. Each compares or composes patterns differently.

Project 1

Streaming Chat - 3 ways

Same LLM chat exposed via polling, SSE, and WebSocket side-by-side. Feel the UX difference.

Project 2

Webhook Dashboard

Stripe-style webhook intake (HMAC, dedup, fast 200) + receiving-side polling vs SSE comparison.

Project 3

LiveOrder Capstone

All 4 patterns in one mini-app: order tracking (SSE), driver chat (WS), payment (webhook), revenue report (polling), AI recommender (SSE/LLM).

Lab folders + Postman collection

Read the code, run it, click Send, see the wire.

- examples/01..07 - one tiny self-contained folder per topic with server.py + client.py
- examples/qa.sh - runs every example end-to-end (49 assertions, all green)
- postman/ - 53 requests in 9 folders, with pre-request scripts for HMAC + JWT
- diagrams/ - 14 technical + 10 persona scenario diagrams, all generated from Python builders
- concepts/ - 8 deep-dive markdown docs (~2900 lines) as the workbook companion
- TEACHING.md - the live-class playbook with talking points per slide

Get the kit

Clone, run the QA, fork the project that fits what you're building.

github.com/fnusatvik07/real-time-events

```
git clone https://github.com/fnusatvik07/real-time-events
cd real-time-events
python -m venv .venv && source .venv/bin/activate
uv pip install fastapi 'uvicorn[standard]' httpx websockets python-dotenv openai
cd examples && bash qa.sh      # 49 checks, ~2 minutes
```

What to remember

Print this on a sticky note.

POLLING

Slow checks, rare events.
OK to be a few seconds
late.

WEBHOOKS

External trigger.
Sign + dedup + return 200
fast.

SSE

Server -> client streaming.
Default for live UI.

WEBSOCKETS

Both ways or binary only.
Reach last.

Default SSE. WebSocket only when bidirectional. Polling for slow. Webhooks for external.

Questions?

Bring your own use case. We'll map it to a pattern.

`github.com/fnusatvik07/real-time-events`